Safe CPU Oversubscription: One Size Does Not Fit All

Jaylen Wang*, Fiodar Kazhamiaka*, Pulkit Misra†, Daniel S. Berger†, Akshitha Sriraman*
*Carnegie Mellon University †Microsoft

1 Introduction

Cloud providers must service a demand of Virtual Machines (VMs), which each request a number of Virtual Processors (VPs). Traditionally, providers support that demand by giving each VP its own Logical Processor (LP), i.e., a hardware thread on the host CPU, to run on. However, VMs tend to underutilize their LPs: prior studies show fleet-average VM CPU utilization around 10% [5, 14]. To exploit this low utilization, providers *oversubscribe* their hosts [4, 5, 14], packing VMs such that there are more VPs than LPs.

Oversubscription lowers the Total Cost of Ownership (TCO) and carbon emissions of serving these VMs [5, 11, 14], but introduces a potential performance overhead: *CPU Wait Time* (CWT), the fraction of time a VP is runnable but unscheduled because no LP is free [4]. Without oversubscription, every VP has its own LP and almost never waits to run; oversubscription removes that guarantee [14]. Thus, as noted in prior work [4, 5, 14], CWT is the Quality-of-Service (QoS) metric providers must reduce when oversubscribing.

To reduce CWT, providers and prior work largely use a static CPU utilization cap and stop admitting VMs to a host once the cap is met; reported caps span 50% [14] to 100% [4, 10]. Such caps are appealing for their simplicity, and at Azure, they have largely kept CWT within QoS targets in part because effective per-host oversubscription is well below what these caps would suggest if applied to every VM in the fleet. In practice, only a subset of VMs is eligible for oversubscription, and oversubscription policies that decide whether to add a VM to a host place limits on how much a host can be oversubscribed (e.g., 15% [5]). Also, prior works [10, 14] evaluate oversubscription policies in simulation, without measuring the fine-grained VP-level effects that affect CWT. Indeed, telemetry at Azure shows CWT varies widely across oversubscribed hosts at the same CPU utilization: even at 10–20% utilization, well below the cap, the p99 CWT is ~5× the p50, motivating us to ask how high the cap can safely be set and what it actually depends on.

To answer this question, we begin from CWT's definition: a VP waits when it is runnable and no LP it can use is free. Whether it waits depends on: (1) how many LPs are available for the VP and (2) how many other VPs want to run on them at the same time. The first is determined by the *platform configuration*: the server environment a VM runs on, specifically the number of LPs and any scheduling constraints that limit which LPs a VP can run on. The second is determined by *workload behavior*: how a VM runs, i.e., the application in the VM and when/for how long its VPs run. We study these factors with experiments spanning controlled benchmarks and real applications, measuring how CWT scales with host utilization. We find that platform-configuration factors (disabling simultaneous multithreading (SMT)) increase the safe oversubscription limit by up to 2.2× and workload-behavior factors (across applications) by up to 1.5×.

Takeaway 0. The **safe amount of oversubscription depends on both platform configuration and workload behavior**, so a single static host utilization cap can either be overly conservative, forgoing TCO/carbon savings, or unsafe, risking QoS violations.

In summary, we contribute:

- The identification of *platform configuration* and *workload behavior* as the two factor families that influence the safe oversubscription limit.
- A characterization of how CWT scales across both factor families, quantifying how each shifts the safe oversubscription limit.
- A demonstration that properly accounting for the safe oversubscription limit can change TCO/carbon-minimizing resource-management choices.

2 Experimental Setup

Hardware. All experiments run on a server with a 28-core Intel Xeon Gold 5512U processor [9] and 128 GB DRAM. The host runs Linux as the hypervisor, as Linux exposes mechanisms to inspect and vary scheduler behavior [17]. A second server acts as the client, sending requests to the VMs at a target rate with Poisson-distributed inter-arrival times.

Scope and metrics. Each experiment runs a fixed set of VMs on a fixed *LP pool* (the set of LPs that VPs are eligible to be scheduled on), and we report the *oversubscription ratio* as total VPs across VMs divided by the pool's LPs (e.g., 16 VPs on 8 LPs is 2×). With SMT enabled, one physical core exposes two sibling LPs. Our QoS metric is CWT; we use 1% CWT, a common target at Azure, throughout. We define *host utilization* as the busy CPU time across all VPs in the pool, divided by LP-pool capacity. For example, two 8-VP VMs each running at 20% per-VP utilization on an 8-LP pool (a 2× oversubscription ratio) produce 40% host utilization.

Measurement. Each data point is the mean of three trials with standard-deviation bars. We measure each VM's CWT and the host utilization. We also collect kernel-scheduler event traces that record when each VP is scheduled to run on or removed from an LP, letting us record VP-level demand that standard CPU utilization metrics smooth over.

3 CWT Curve and Scheduling Constraints

To measure CWT vs. host utilization, we sweep per-VM CPU utilization on two 8-VP VMs running a Go HTTP server and sharing an 8-LP pool at a 2× oversubscription ratio.

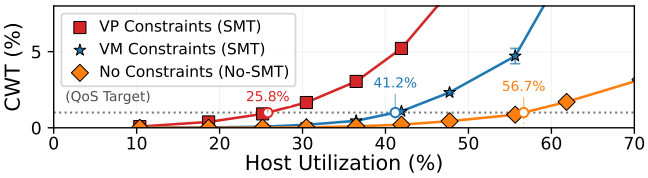

Figure 1. Max CWT across two VMs vs. host utilization for a Go HTTP server at 2× oversubscription on an 8-LP pool, under SMT with *VP constraints* (default), *VM constraints*, and *no-SMT*. Markers show where each curve first meets the 1% CWT target; the safe oversubscription limit increases 1.6× (*VM constraints*) and 2.2× (*no-SMT*) above default.

What makes the setup mirror that of a real public cloud is its scheduling regime: providers restrict which VPs may co-run on the sibling LPs of one physical core to prevent side-channel attacks through the core's shared microarchitectural state, both across tenants and within a single VM (e.g., between an untrusted user thread and a kernel thread) [1, 12].

The cloud default, including at Azure, is SMT with the strictest such constraint, *VP constraints*: sibling LPs may host only VPs that are *guest siblings* (i.e., VPs the guest OS sees as siblings) in the same VM [1, 7, 12]. We measure that default first, then compare two looser cases: *VM constraints*, where sibling LPs may host any two VPs of one VM but not different VMs, and *no-SMT*, where each physical core exposes only one LP. We run all three regimes with an 8-LP pool. *VM constraints* are not a deployment option (providers do not accept intra-VM side-channel risk [7, 12]); we include them to isolate the overhead from the cross-VM rule.

Fig. 1 plots host utilization vs. the max CWT across the two VMs for the three regimes, marking where each curve first meets the CWT target. The 1% CWT target is met at 25.8% host utilization, well below prior work's static caps [4, 5, 14].

We convert where the CWT target is met into the *safe oversubscription limit* (the max oversubscription ratio that keeps CWT below the target) by dividing its host utilization by the per-VM CPU utilization. Thus, at the fleet-typical 10% VM utilization, we find a safe oversubscription limit of only 2.58×, well below the 5× and 10× that 50% and 100% caps allow. This gap is largely due to short bursts of VP utilization, which the per-second utilization used by prior caps [4, 14] smooths over; we study the mechanism in §5.

Takeaway 1. At 10% per-VM CPU utilization, the safe oversubscription limit is only ≈ 2.6×, so **static utilization caps used in practice today allow up to ≈4× more oversubscription than the CWT QoS target permits, motivating an update of such caps.**

Looser scheduling constraints shift the curve. On the same hardware and application, the safe oversubscription limit increases from 2.6× under the *VP constraints* default to 4.1× with *VM constraints* and 5.7× with *no-SMT*, 1.6× and 2.2× higher than the default, respectively. The difference is due to VP placement restrictions: under *VP constraints*, if a VP is running and its guest sibling is not, the sibling LP on that core cannot host a different VP, reducing the usable LP capacity and causing CWT. *VM constraints* and *no-SMT* progressively widen the set of allowed VP co-runners. The gap between *VP constraints* and *no-SMT* holds across all 13 applications we measure, spanning cloud application classes identified by prior work [8], from databases [13] to inference serving [15]. The no-SMT-vs-VP-constraints ratio of safe oversubscription limits ranges from 1.8× to 3.2×.

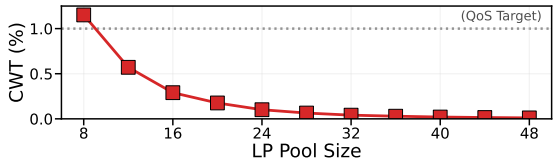

Figure 2. Mean CWT vs. LP pool size under SMT with VP constraints, keeping host utilization (20%) and oversubscription ratio (2×) fixed. CWT exceeds the 1% target only at 8 LPs (1.15%) and drops to 0.01% by 48 LPs.

Takeaway 2. Relaxing VP-to-VM constraints increases the safe oversubscription limit 1.6× and **disabling SMT, due to differences in scheduling constraints, can increase safe oversubscription limits by ~2×.**

4 LP Pool Size

LP pool size is another platform-configuration factor that can change CWT. Across the fleet, hosts range in core counts and thus range in LP-pool sizes. That variation raises the question of whether a cap calibrated on one host size transfers to another. To isolate how LP pool size affects CWT, we run the Go HTTP server while fixing all other parameters, only varying LP pool size. The number of VMs grows from 2 to 12, as the LP pool grows from 8 to 48 LPs. We fix host utilization at 20%, oversubscription ratio at 2×, and use SMT with *VP constraints*. Fig. 2 plots LP pool size vs. mean CWT.

The curve decays roughly exponentially as the LP pool grows. Doubling the LP pool from 8 to 16 LPs reduces CWT ≈ 4×, and increasing to 48 LPs cuts it to 0.01%.

Takeaway 3. All else equal, **larger LP pools can support a higher safe oversubscription limit than smaller ones**, so a cap calibrated on one host size may be unsafe or overly conservative for other host sizes.

5 Workload Behavior

We now focus on workload-behavior factors that describe how a VM runs: which application runs and how it uses its VPs. We fix the platform configuration using the same parameters as §3, and ask whether the safe oversubscription limit changes across applications.

For the 13 applications we study, the 1% CWT target is met at host utilization between 16% and 26%, a 1.6× difference. At the fleet-typical 10% per-VM CPU utilization, this difference corresponds to safe oversubscription limits from

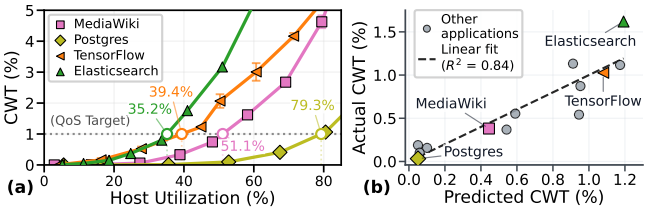

Figure 3. (a) CWT vs. host utilization for four representative applications under *no-SMT* with platform configuration held fixed. (b) Predicted vs. measured CWT at 40% host utilization for 13 applications from wide-burst prevalence and duration. VP-coordination features account for much of the cross-application variation in CWT.

1.7× for Elasticsearch [6] to 2.6× for a Go HTTP server [16]. Figure 3(a) plots max CWT vs. host utilization under *no-SMT* for four representative applications, where the 1% target is met at host utilization between 35.2% and 79.3%. At 40% host utilization, CWT ranges from 0.03% to 1.62% (50× variation).

Takeaway 4. Even with platform configuration held fixed, the safe oversubscription limit varies up to 1.6× across applications, so **cap calibration must account for what runs in the VM, not just where it runs.**

VP coordination explains much of the application variation. What about an application drives this variation in CWT? As this variation exists when applications operate at the same per-second host utilization, the answer lies at a finer time-scale than utilization averages capture. To get finer-grained visibility, we turn to our kernel-scheduler traces, which show that applications differ in their *VP coordination*: in some applications, a VM's VPs tend to run together; in others, they run at different times. This fine-grained coordination of VPs directly shapes CWT: when VMs' VPs tend to run in an uncoordinated fashion, there is more freedom to schedule the VPs onto the LPs.

To test whether VP coordination explains the variation, we extract two application features built on a *wide burst*: an interval where most of a VM's VPs (we use ≥75%) are simultaneously runnable, contending for the LP pool. The first is wide-burst prevalence: the fraction of time the VM is in a wide burst. The second is wide-burst duration: how long each wide burst lasts on average. Even at the same host utilization, applications can differ along both features, since per-second utilization aggregates over both VP count and time. Frequent wide bursts spike instantaneous demand for many LPs, and long wide bursts hold that demand in the LP pool long enough for other runnable VPs to wait.

To test the hypothesis, we fit a linear regression of each application's CWT on its wide-burst prevalence and wide-burst duration. We plot the regression against the measured CWT for the 13 applications at 40% host utilization under *no-SMT* in Figure 3(b). CWT increases with both features: applications whose VPs more often enter a wide burst, and stay in one longer, have higher CWT. The fit reaches $R^2 = 0.84$, so the two features account for most of the cross-application variation that host utilization does not capture.

The regression shows correlation; to test causality, we use the same setup but use a synthetic Go HTTP server whose requests use a configurable number of VPs in parallel. At 40% host utilization, increasing the number of VPs used per request from 1 to all 8 increases CWT from 1.4% to 3% (2.1×). Thus, we isolate that VP coordination increases CWT.

Takeaway 5. At fixed host utilization, **CWT depends on whether many VPs in one VM become runnable together and stay runnable**, so per-second utilization alone is not a sufficient indicator of CWT.

6 Future Directions and Conclusion

We have observed other factors with more minor impacts on CWT, including the hypervisor's scheduler implementation and cross-VP synchronization (e.g., an application's locking behavior), which we defer for brevity. Moreover, our experiment setups are deliberately targeted, using two VMs running the same application. However, preliminary sweeps over heterogeneous applications, VM sizes, hardware platforms, and other hypervisors suggest the same qualitative trends, with full quantification left to future work.

Within the factors we quantify, the variability in the safe oversubscription limit reveals implications for oversubscription policy and platform design. For oversubscription policy, the variability motivates rethinking the current static-cap approach: policies should be aware of varying safe limits, take advantage of cases that permit more oversubscription, and safely handle cases that permit less. Finding practical policies that do so remains an open challenge, given providers' limited visibility into fine-grained VM behavior at scale.

For resource management (e.g., platform design and VM placement), accounting for the differences in safe oversubscription limits can change which design choices minimize TCO and carbon. We have done an initial analysis of one such choice: enabling vs. disabling SMT. Traditional TCO/carbon analyses favor enabling SMT [2, 3], as it doubles LPs per physical core, but §3 shows *no-SMT* can increase the safe oversubscription limit by 2.2×. Combining our experimental results with an internal TCO/carbon model (accounting for other factors, e.g., power, performance), our analysis suggests disabling SMT's gain in safe oversubscription is enough to potentially make it the lower-carbon and TCO option.

More broadly, accounting for our identified factors may shift other resource-management decisions, e.g., whether to use higher-core-count CPUs with larger LP pools or how to place high-VP-coordination VMs onto hosts.

Takeaway 6. **Safe oversubscription is a design variable, not a fleet-wide constant**: platform configuration and workload behavior can change which resource-management choices minimize TCO and carbon.

References

[1] Amazon Web Services. 2026. The EC2 Approach to Preventing Side-Channels. https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/the-ec2-approach-to-preventing-side-channels.html. Accessed: May 17, 2026.

[2] AMD. 2025. No Compromise: Driving Server Performance and Efficiency with AMD EPYC and SMT. https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/amd-epyc-smt-technology-brief.pdf. Accessed: May 19, 2026.

[3] Ampere Computing. 2022. Looking Beyond SMT in the Cloud. https://amperecomputing.com/blogs/looking-beyond-smt-in-the-cloud. Accessed: May 19, 2026.

[4] Noman Bashir, Nan Deng, Krzysztof Rzadca, David Irwin, Sree Kodak, and Rohit Jnagal. 2021. Take It to the Limit: Peak Prediction-Driven Resource Overcommitment in Datacenters. In *European Conference on Computer Systems*.

[5] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. 2017. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. In *Symposium on Operating Systems Principles*.

[6] Elastic. 2026. Elasticsearch. https://www.elastic.co/elasticsearch. Accessed: May 16, 2026.

[7] Google Cloud. 2026. CPU Platforms. https://cloud.google.com/compute/docs/cpu-platforms. Accessed: May 17, 2026.

[8] Lexiang Huang, Anjaly Parayil, Jue Zhang, Xiaoting Qin, Chetan Bansal, Jovan Stojkovic, Pantea Zardoshti, Pulkit Misra, Eli Cortez, Raphael Ghelman, Iñigo Goiri, Saravan Rajmohan, Jim Kleewein, Rodrigo Fonseca, Timothy Zhu, and Ricardo Bianchini. 2025. Workload Intelligence: Workload-Aware IaaS Abstraction for Cloud Efficiency. In *International Conference for High Performance Computing, Networking, Storage and Analysis*.

[9] Intel. 2023. Intel Xeon Gold 5512U Processor. https://ark.intel.com/content/www/us/en/ark/products/237565/intel-xeon-gold-5512u-processor-52-5m-cache-2-10-ghz.html. Accessed: May 19, 2026.

[10] Seyedali Jokar Jandaghi, Kaveh Mahdaviani, Amirhossein Mirhosseini, Sameh Elnikety, Cristiana Amza, and Bianca Schroeder. 2024. AUDIBLE: A Convolution-Based Resource Allocator for Oversubscribing Burstable Virtual Machines. In *International Conference on Architectural Support for Programming Languages and Operating Systems*.

[11] Jialun Lyu, Jaylen Wang, Kali Frost, Chaojie Zhang, Celine Irvene, Esha Choukse, Rodrigo Fonseca, Ricardo Bianchini, Fiodar Kazhamiaka, and Daniel S. Berger. 2023. Myths and Misconceptions Around Reducing Carbon Embedded in Cloud Platforms. In *Workshop on Sustainable Computer Systems*.

[12] Microsoft. 2026. About Hyper-V Hypervisor Scheduler Type Selection. https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/manage/about-hyper-v-scheduler-type-selection. Accessed: May 17, 2026.

[13] PostgreSQL Global Development Group. 2026. PostgreSQL. https://www.postgresql.org/. Accessed: May 16, 2026.

[14] Benjamin Reidys, Pantea Zardoshti, Iñigo Goiri, Celine Irvene, Daniel S. Berger, Haoran Ma, Kapil Arya, Eli Cortez, Taylor Stark, Eugene Bak, Mehmet Iyigun, Stanko Novakovic, Lisa Hsu, Karel Trueba, Abhisek Pan, Chetan Bansal, Saravan Rajmohan, Jian Huang, and Ricardo Bianchini. 2025. Coach: Exploiting Temporal Patterns for All-Resource Oversubscription in Cloud Platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems*.

[15] TensorFlow Authors. 2026. TensorFlow. https://www.tensorflow.org/. Accessed: May 16, 2026.

[16] The Go Authors. 2026. net/http. https://pkg.go.dev/net/http. Accessed: May 16, 2026.

[17] The Linux Kernel. 2026. Core Scheduling. https://docs.kernel.org/admin-guide/hw-vuln/core-scheduling.html. Accessed: May 20, 2026.